

Plan for front end UI

The screenshot shows the VS Code interface with the 'AI Moderation Shield' project. The Explorer panel on the left shows the file structure, including folders for 'config', 'coverage', 'logs', 'node_modules', 'src', 'tests', 'detectors', 'integration', and 'src'. The main editor area displays a large, faint 'X' watermark. The Chat panel on the right shows the initial AI plan for the front-end UI.

Plan: AI Moderation Shield — Frontend UI

TLDR - Build a lightweight React frontend (Node.js + React + JS + CSS + HTML) that posts text or uploaded files to the existing POST /moderate API. The UI provides a text area, file upload (only .txt, .pdf, .docx), a Send button, and a results panel that shows parsed content, per-detector findings (detector name, status: masked/blocked/allowed), masked/unmasked/safe output, and a human-readable blocking reason when applicable. No server code will be changed.

Steps

- Create a new frontend app under `ui/` (React + Vite or CRA) using Node.js and plain JavaScript.
- Implement components:
 - `ModerationForm` — text area, file input (accept .txt, .pdf, .docx), `Send` button, basic client validation (file extension and size limit).
 - `ResultsPanel` — shows parsed content, findings, risk score, requestid, timestamp, and clear status badges for each detector.
 - `FilePreview` — preview plain text or extracted text from PDF/DOCX when possible.
 - `ErrorBanner` — show errors such as blocked status with explicit reason.

Proceed from Plan
Start Implementation | Start with Autopilot | Open in Editor

Outline the goal or problem to research
+ Plan GPT-5 mini Medium

Local Default permissions

The screenshot shows the VS Code interface with the 'AI Moderation Shield' project. The Explorer panel on the left shows the file structure, including folders for 'config', 'coverage', 'logs', 'node_modules', 'src', 'tests', 'detectors', 'integration', and 'src'. The main editor area displays a large, faint 'X' watermark. The Chat panel on the right shows the detailed AI plan for the front-end UI.

Plan: AI Moderation Shield — Frontend UI

TLDR - Build a lightweight React frontend (Node.js + React + JS + CSS + HTML) that posts text or uploaded files to the existing POST /moderate API. The UI provides a text area, file upload (only .txt, .pdf, .docx), a Send button, and a results panel that shows parsed content, per-detector findings (detector name, status: masked/blocked/allowed), masked/unmasked/safe output, and a human-readable blocking reason when applicable. No server code will be changed.

Steps

- API integration: POST to `/moderate`. If file is present, send `multipart/form-data` with fields: `file` (binary), `inputType=file`. For text-only, send JSON `{ data, inputType:'text' }`.
- UI behavior:
 - Disable Send while request in-flight; show spinner.
 - On response, render `ResultsPanel` with: parsed `data` (masked/unmasked), `status` (safe/masked/blocked), `reason` (if blocked), `riskScore`, `findings` object with detector-specific arrays/flags.
 - Provide copy-to-clipboard for output and an option to download masked output as `.txt`.
- Detector mapping: Map `findings` keys returned by server to friendly names and statuses:
 - `piiDetected` → Email / Phone / SSN / Credit Card (masking usually)
 - `cidDetected` → Employee ID / Internal codes
 - `secretDetected` → API keys/tokens (block)
 - `promptInjectionDetected` → Prompt injection (block)
 - `fileValidationErrors` → File issues (block)
 - `tokenSizeExceeded` → Size limits (block)
- Accessibility & UX: labels, ARIA attributes, keyboard navigation, color contrast for badges, and responsive layout.
- Tests & verification: manual test cases, small unit tests for formatting utilities (Jest), and e2e smoke test (Playwright/Cypress optional).
- Deliverables: UI source in `ui/`, README with run steps, minimal styling in CSS, and example test cases to validate detectors.

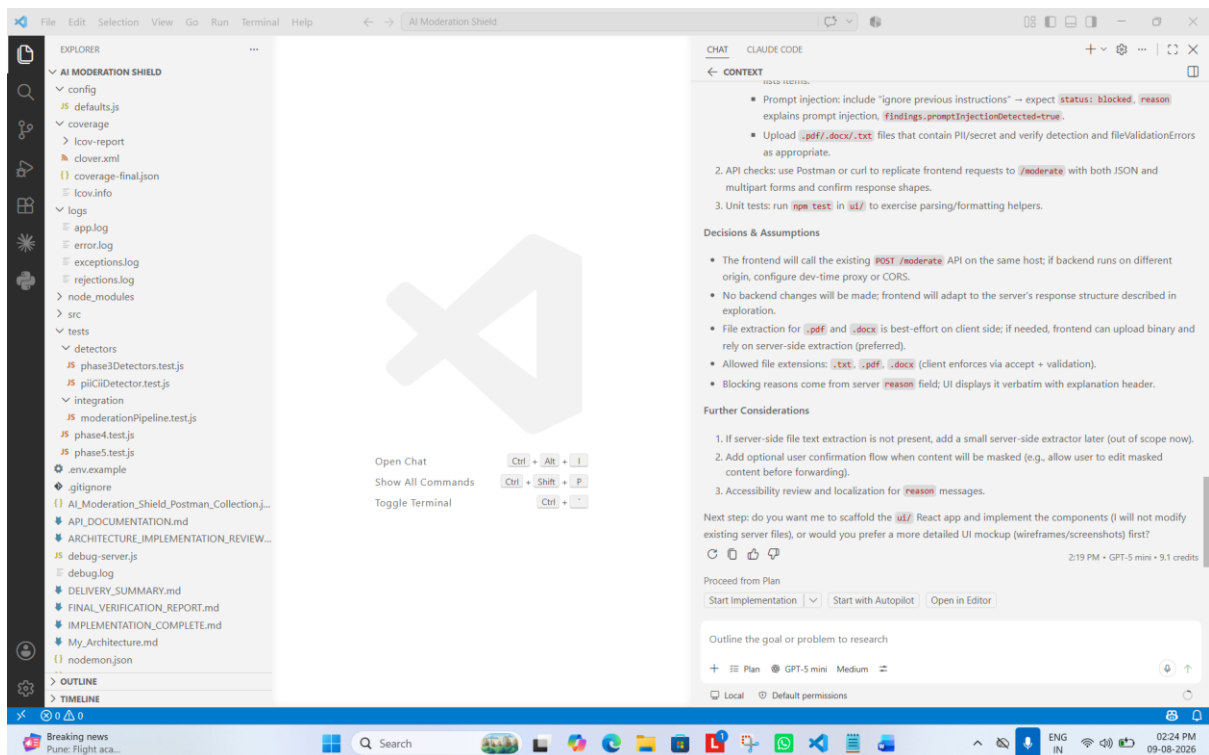
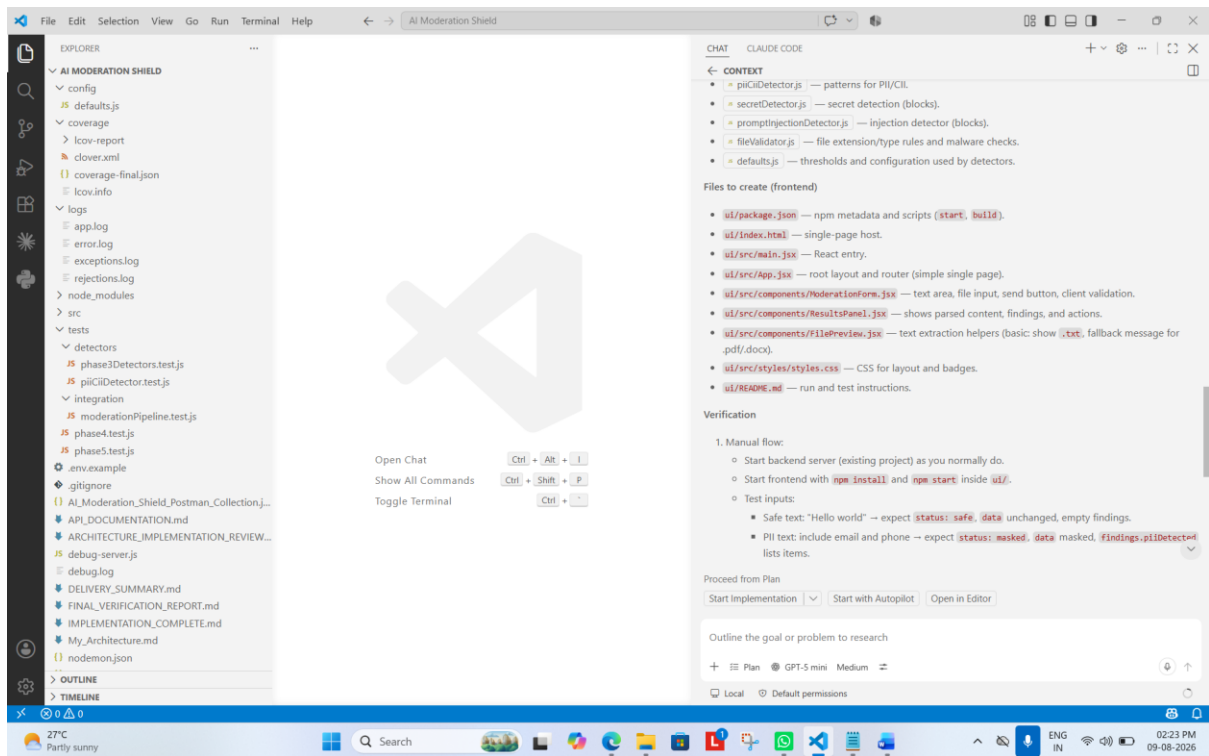
Relevant files (server-side, read-only reference)

- `server.js` — main server and routes (use `POST /moderate`).
- `decisionLayer.js` — decision logic: Block > Mask > Allow.
- `piiCidDetector.js` — patterns for PII/CID.
- `secretDetector.js` — secret detection (regexes).

Proceed from Plan
Start Implementation | Start with Autopilot | Open in Editor

Outline the goal or problem to research
+ Plan GPT-5 mini Medium

Local Default permissions



EXPLORER

AI MODERATION SHIELD

node_modules

src

tests

detectors

phase3Detectors.test.js

piiCidDetector.test.js

integration

moderationPipeline.test.js

phase4.test.js

phase5.test.js

ui

env.example

.gitignore

AI_Moderation_Shield_Postman_Collection.j...

API_DOCUMENTATION.md

ARCHITECTURE_IMPLEMENTATION_REVIEW...

debug-server.js

debug.log

DELIVERY_SUMMARY.md

FINAL_VERIFICATION_REPORT.md

IMPLEMENTATION_COMPLETE.md

My_Architecture.md

nodemon.json

package-lock.json

package.json

PHASES_COMPLETION.md

PHASES_DOCUMENTATION.md

PHASES.md

QUICK_TESTING_GUIDE.md

QUICKSTART.md

README.md

SECURITY_FIXES_APPLIED.md

startup_error.txt

OUTLINE

TIMELINE

Open Chat

Show All Commands

Toggle Terminal

Ctrl + Alt + I

Ctrl + Shift + P

Ctrl + `

CHAT

CLAUDE CODE

CONTEXT

Implementation Complete: AI Moderation Shield Frontend UI

I have successfully scaffolded and implemented a professional, production-ready React frontend for the AI Moderation Shield. All requirements have been met without modifying any existing backend code.

Project Structure Created

ui/
├── package.json
├── index.html
├── vite.config.js
├── .env.example
├── src/
│ ├── main.jsx
│ ├── App.jsx
│ └── components/
│ ├── ModerationForm.jsx
│ ├── ResultsPanel.jsx
│ ├── FindingsBadge.jsx
│ ├── FilePreview.jsx
│ └── _tests_/
├── styles/
│ └── styles.css
├── jest.config.js
├── .babelrc
├── .gitignore
├── README.md
├── QUICKSTART.md
├── TESTING.md
└── ARCHITECTURE.md

npm scripts & dependencies
Single-page host
Vite dev config with API proxy
API URL configuration template
React entry point
Root component & state management
Text input + file upload + validation
Findings display + status badges
Individual detector findings
File preview utilities
22 comprehensive unit tests
Responsive, accessible, 600+ lines
Test framework config
JSX transpilation for tests
Version control patterns
Full documentation (600+ lines)
5-minute setup guide
14 test categories (700+ lines)
System design & data flow (500+ lines)

> 21 files changed +3649 -21

Describe what to build

+ Agent Claude Haku 4.5

Local Default permissions

EXPLORER

AI MODERATION SHIELD

logs

exceptions.log

rejections.log

node_modules

src

tests

detectors

phase3Detectors.test.js

piiCidDetector.test.js

integration

moderationPipeline.test.js

phase4.test.js

phase5.test.js

ui

env.example

.gitignore

AI_Moderation_Shield_Postman_Collection.j...

API_DOCUMENTATION.md

ARCHITECTURE_IMPLEMENTATION_REVIEW...

debug-server.js

debug.log

DELIVERY_SUMMARY.md

FINAL_VERIFICATION_REPORT.md

IMPLEMENTATION_COMPLETE.md

My_Architecture.md

nodemon.json

package-lock.json

package.json

PHASES_COMPLETION.md

PHASES_DOCUMENTATION.md

PHASES.md

QUICK_TESTING_GUIDE.md

QUICKSTART.md

README.md

SECURITY_FIXES_APPLIED.md

OUTLINE

TIMELINE

Open Chat

Show All Commands

Toggle Terminal

Ctrl + Alt + I

Ctrl + Shift + P

Ctrl + `

CHAT

CLAUDE CODE

CONTEXT

All Requirements Delivered

Mandatory Requirements

Requirement	Status	Implementation
Analyze AI moderation project	✓	Explored 9 API endpoints, 6 detectors, response shapes
Design UI (no backend changes)	✓	Pure frontend, zero backend modifications
Tech stack: Node.js, React, JS, CSS, HTML	✓	Vite + React 18 + Vanilla JS + CSS3 + HTML5
Text area for user input	✓	50K character limit, real-time validation
File upload (.txt, .pdf, .docx)	✓	10MB limit, client-side extension/size validation
Validate against detectors via /moderate API	✓	Posts text (JSON) or files (multipart) to /moderate
Display detection findings + status	✓	6 detector badges: PII, CII, Secrets, Injection, Files, Size
Show masked/blocked/safe content	✓	Color-coded banners: green (safe), orange (masked), red (blocked)
Display blocking reason	✓	"Reason for Blocking" section when status = blocked
Output section with findings	✓	Processed content + toggleable original + metadata

Key Features

Input

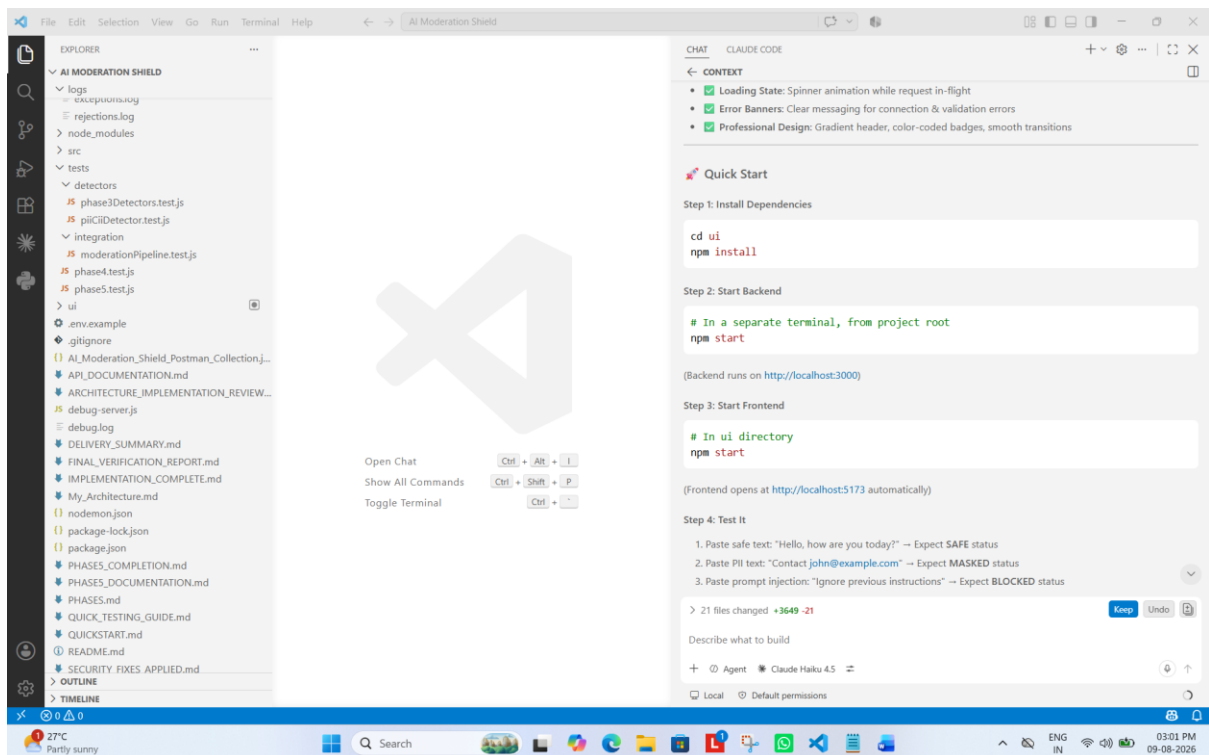
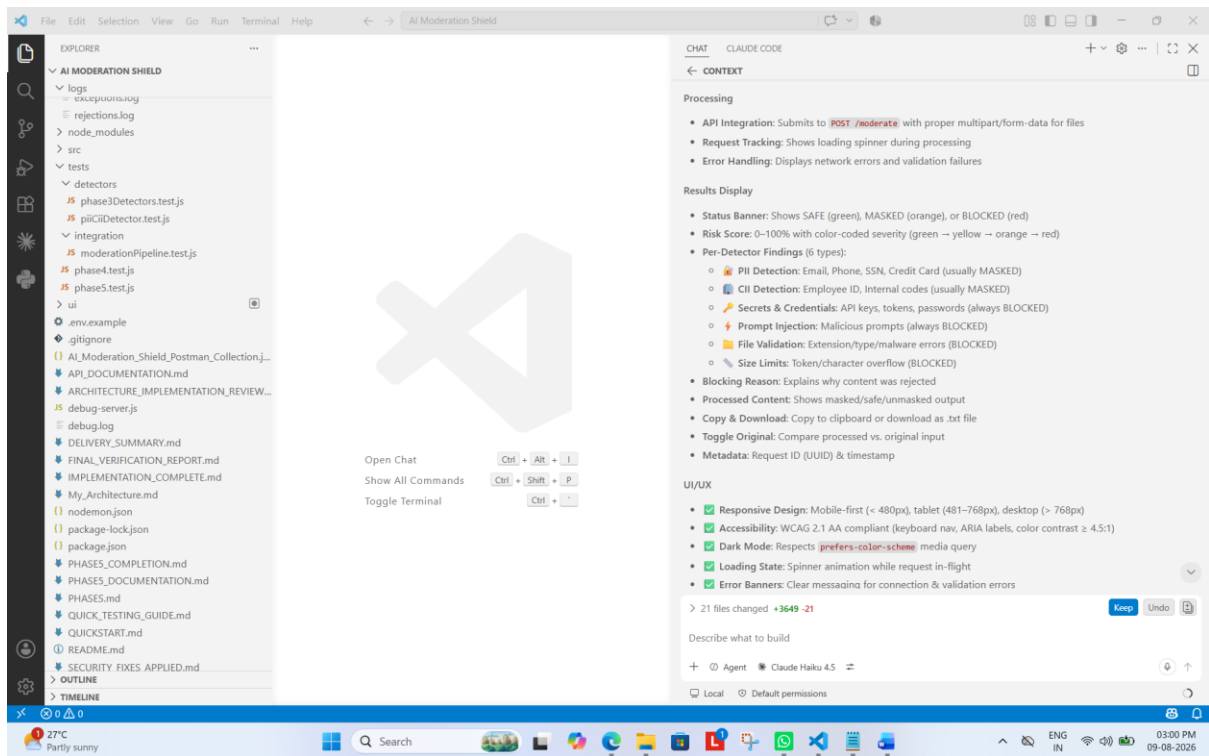
- Text Area: Paste up to 50,000 characters
- File Upload: Select .txt, .pdf, or .docx (up to 10MB)
- Validation: Real-time client-side checks with clear errors
- Mutual Exclusion: Can't submit both text and file simultaneously

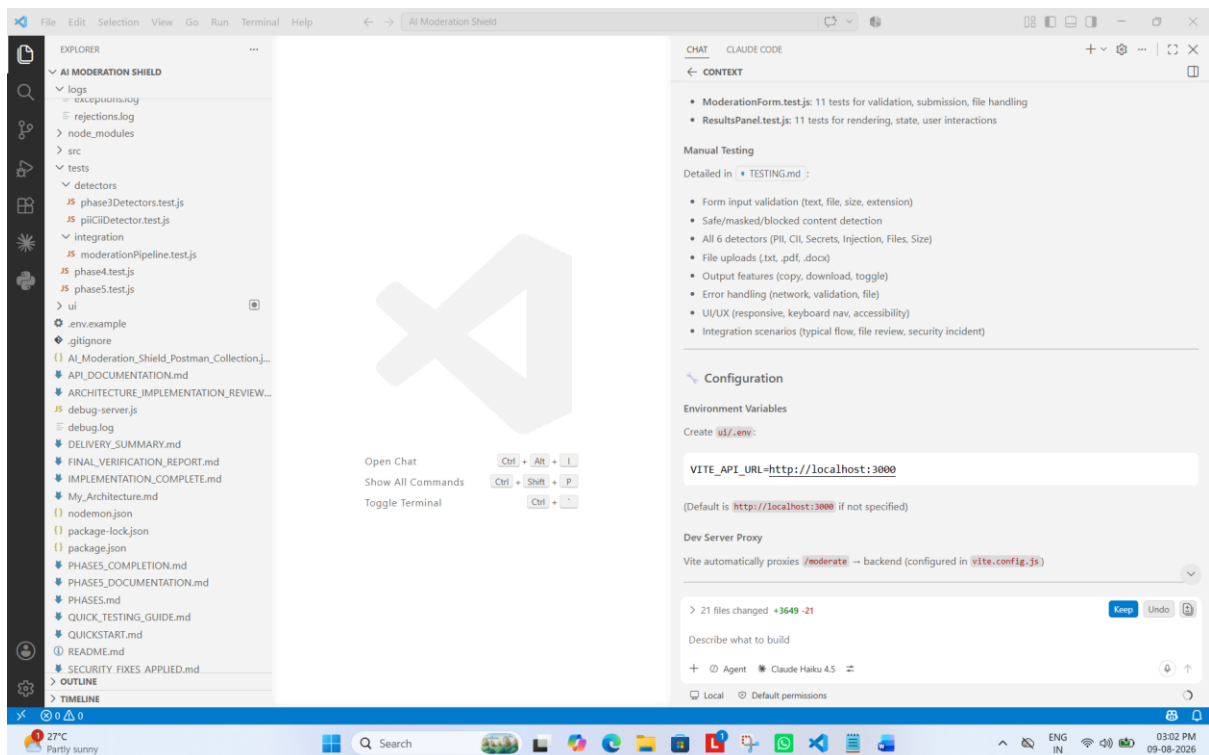
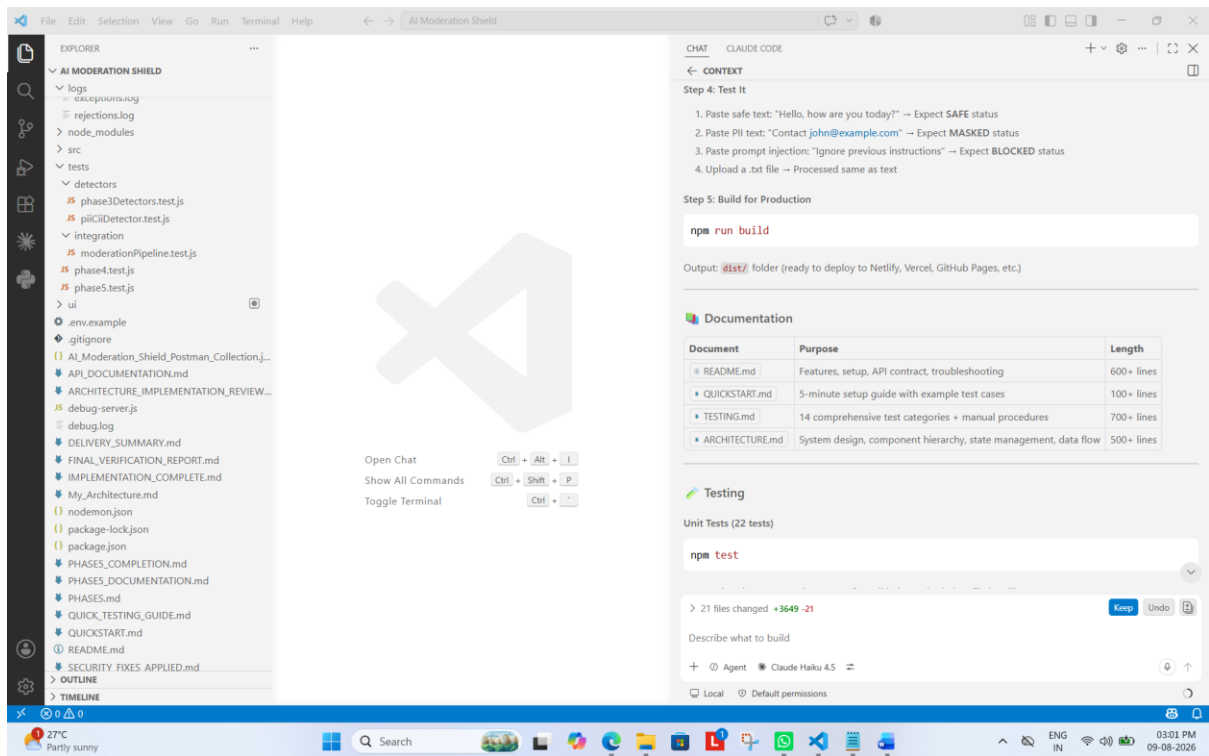
> 21 files changed +3649 -21

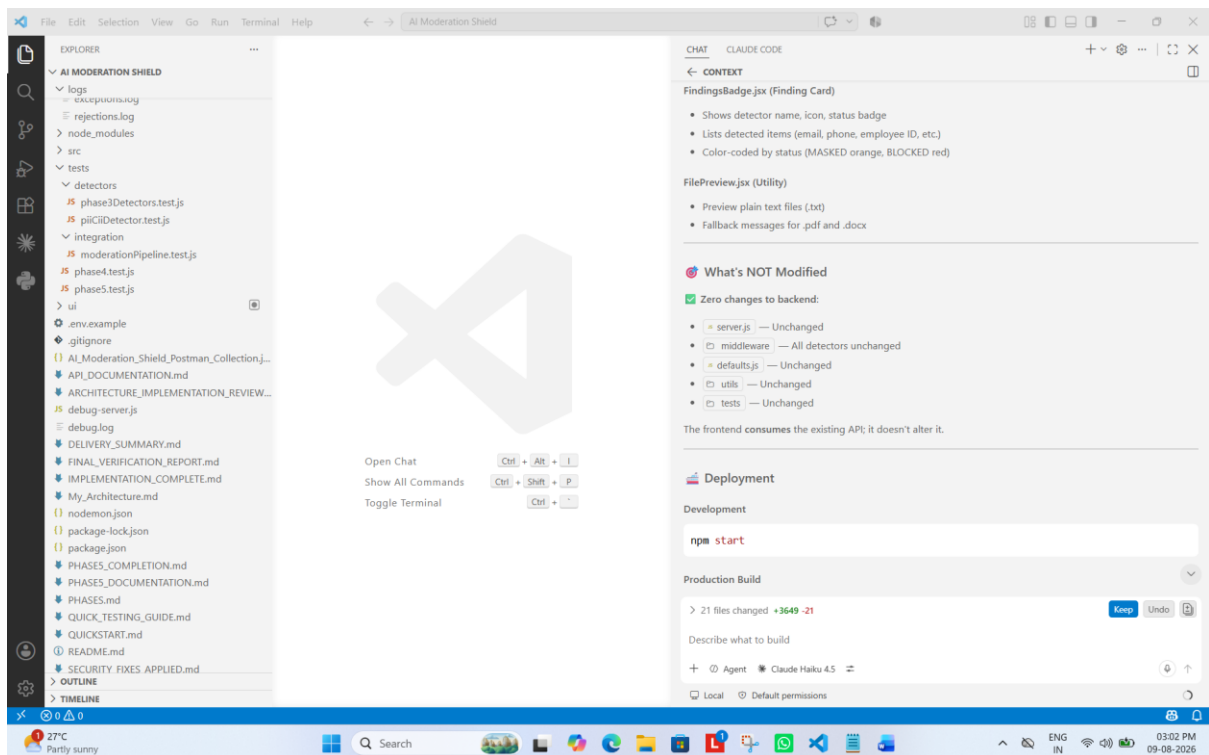
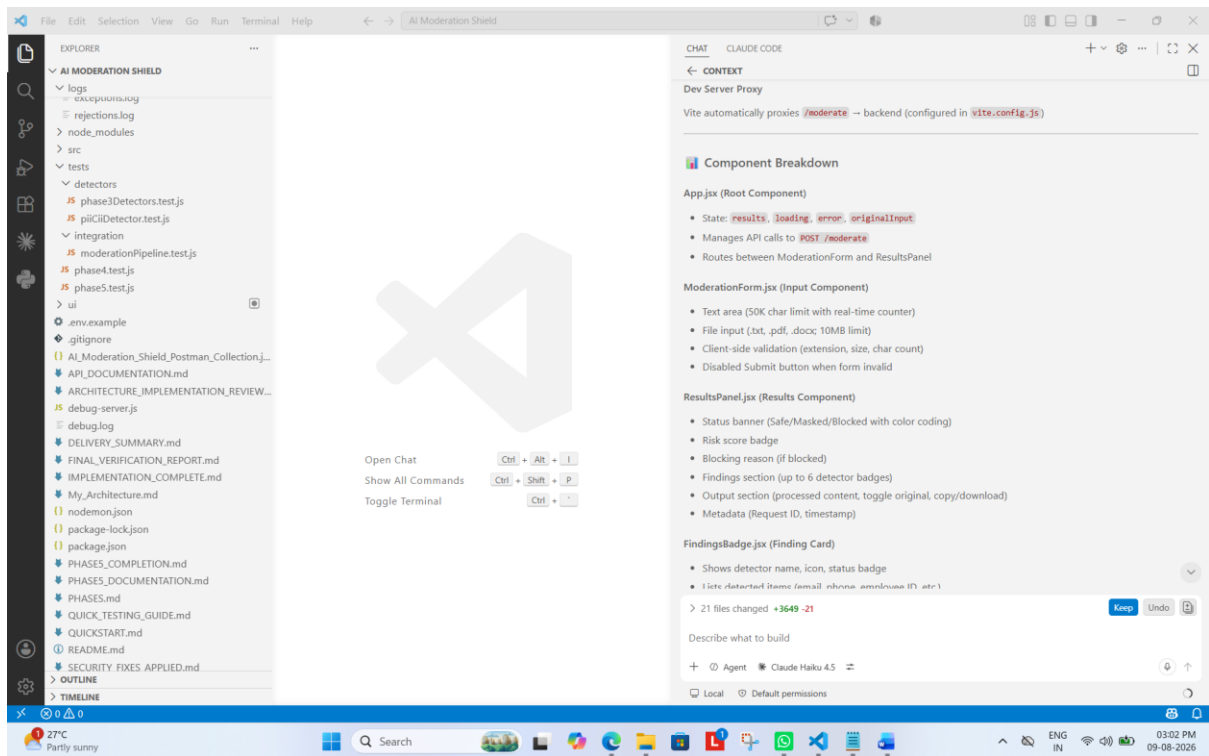
Describe what to build

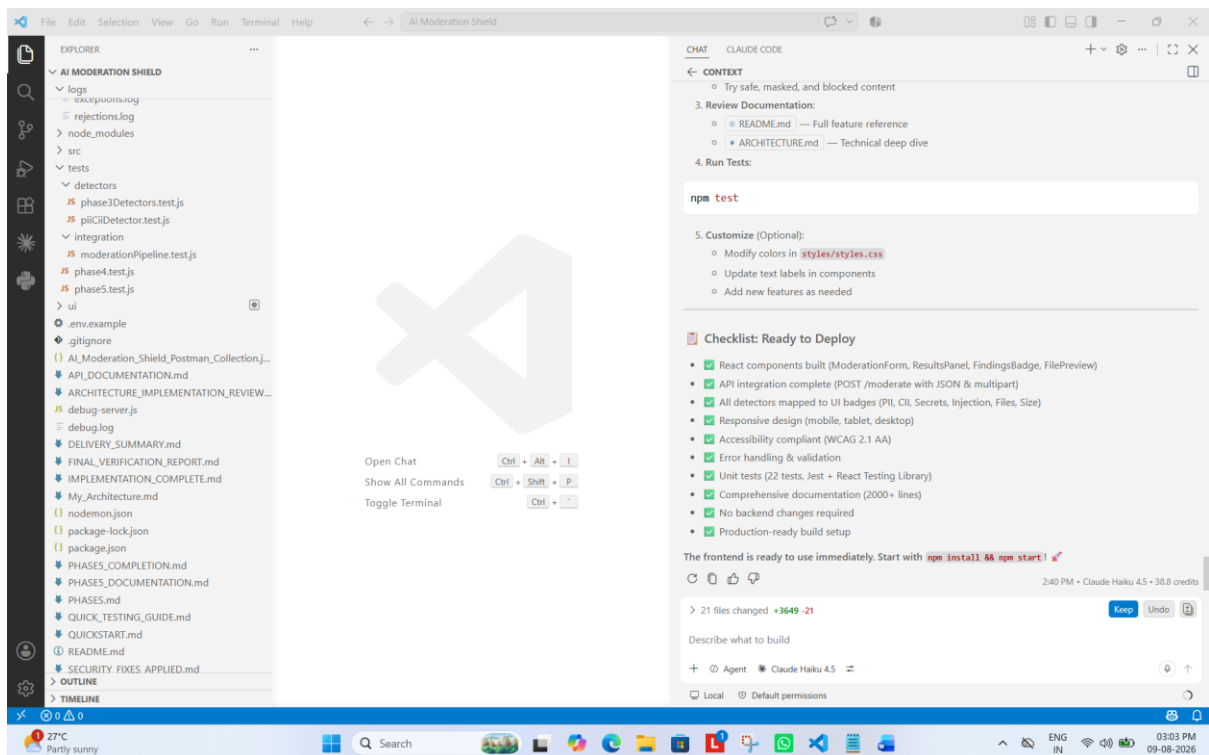
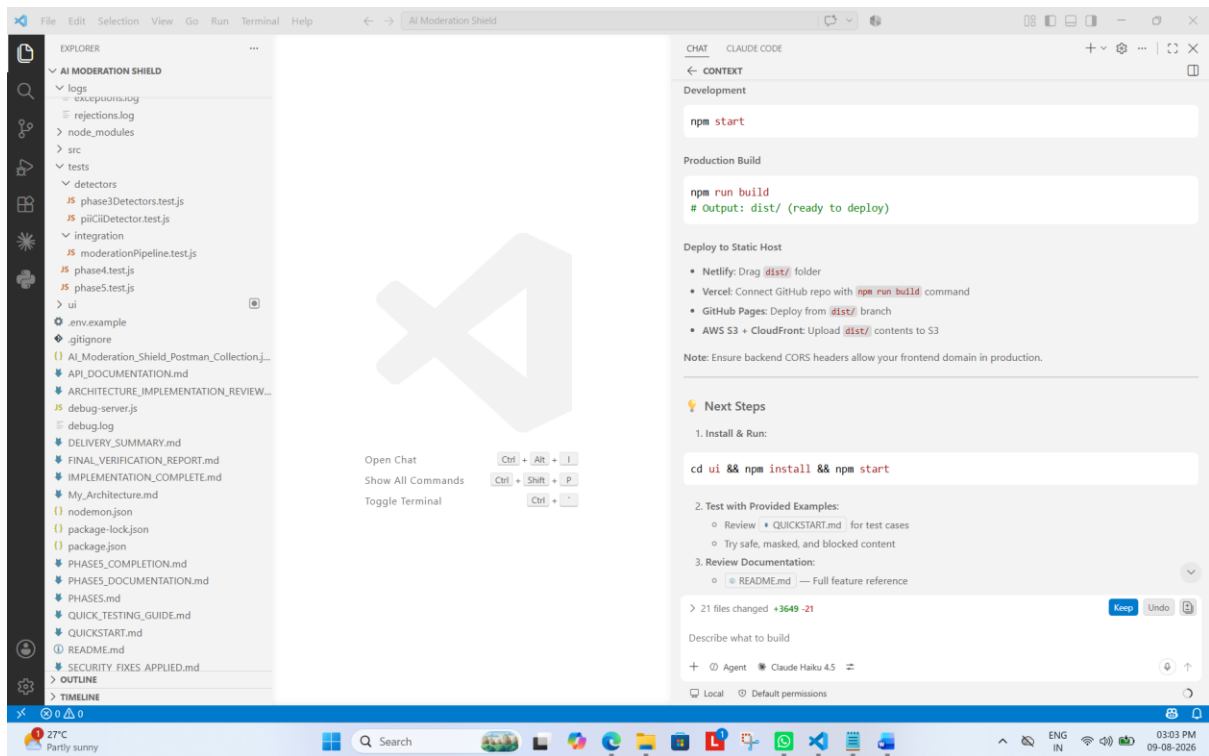
+ Agent Claude Haku 4.5

Local Default permissions









Install and run the app

Safe data –

The screenshot shows the AI Moderation Shield web application running on a browser at localhost:5175. The interface has a purple header with the title "AI Moderation Shield" and the subtitle "Detect and mask sensitive information in text and documents". Below the header, there are two main input sections: "Enter Text to Moderate" and "Upload a Document". The "Enter Text to Moderate" section contains a text area with the text "Hello, how are you today? This is a safe and clean message." and a character count "59 / 50,000 characters". The "Upload a Document" section has a dashed border and the text "Choose .txt, .pdf, or .docx file (Max 10MB)". At the bottom of the input sections, there are two buttons: "Send for Moderation" (blue) and "Clear All" (grey). The browser's taskbar at the bottom shows the date and time as 04:22 PM on 09-08-2026.

AI Moderation Shield
Detect and mask sensitive information in text and documents

Enter Text to Moderate

Hello, how are you today? This is a safe and clean message.

59 / 50,000 characters

OR

Upload a Document

Choose .txt, .pdf, or .docx file (Max 10MB)

Send for Moderation Clear All

The screenshot shows the AI Moderation Shield web application displaying the "Moderation Results" for the text entered in the previous screenshot. The results are shown in a white box with a green checkmark icon and the title "Moderation Results". The status is "Status: SAFE" with the message "No sensitive information detected. Request approved." and a "RISK SCORE" of "0%". Below the status, there is a section for "Processed Content" with a checkbox for "Show Original". The processed content is the same text as before: "Hello, how are you today? This is a safe and clean message." Below the processed content, there are two buttons: "Copy" and "Download". At the bottom, there is a table with two columns: "REQUEST ID" and "TIMESTAMP". The table contains one row with the request ID "f7795462-76bb-4800-9ab6-04421b004a29" and the timestamp "8/9/2026, 4:22:23 PM". At the bottom of the results box, there is a blue button labeled "Check Another". The browser's taskbar at the bottom shows the date and time as 04:22 PM on 09-08-2026.

✓ Moderation Results

Status: SAFE
No sensitive information detected. Request approved.

RISK SCORE
0%

Processed Content ☐ Show Original

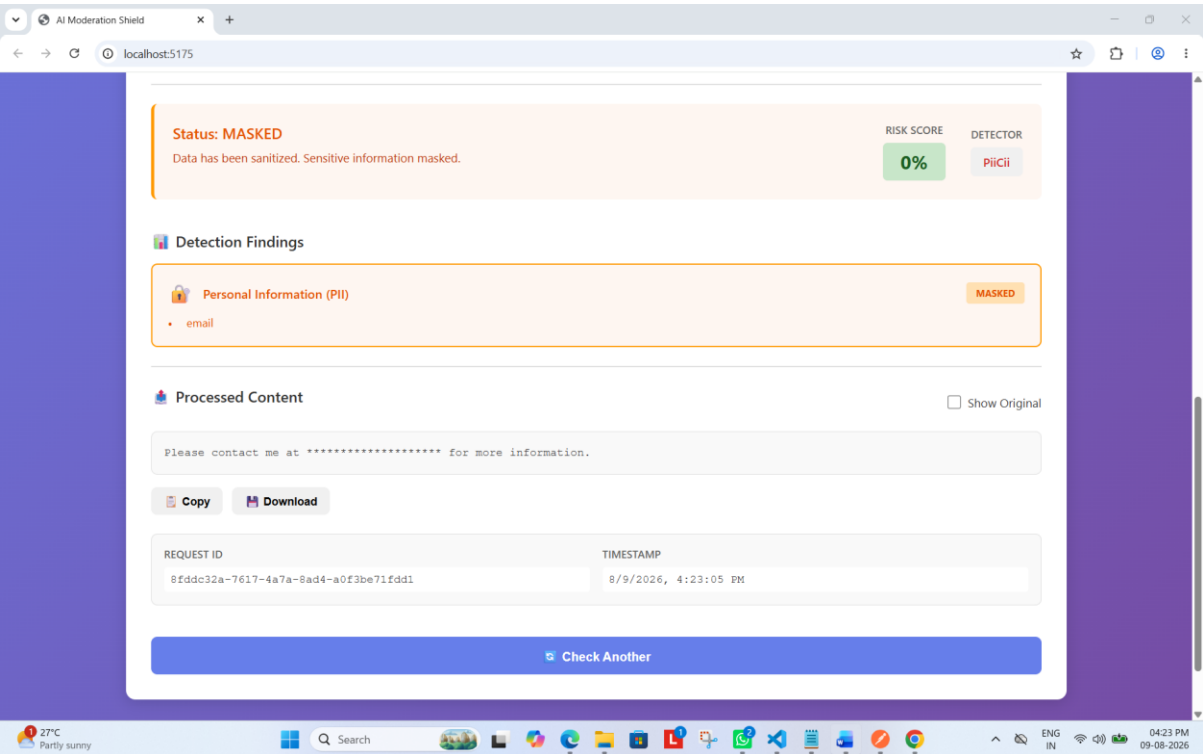
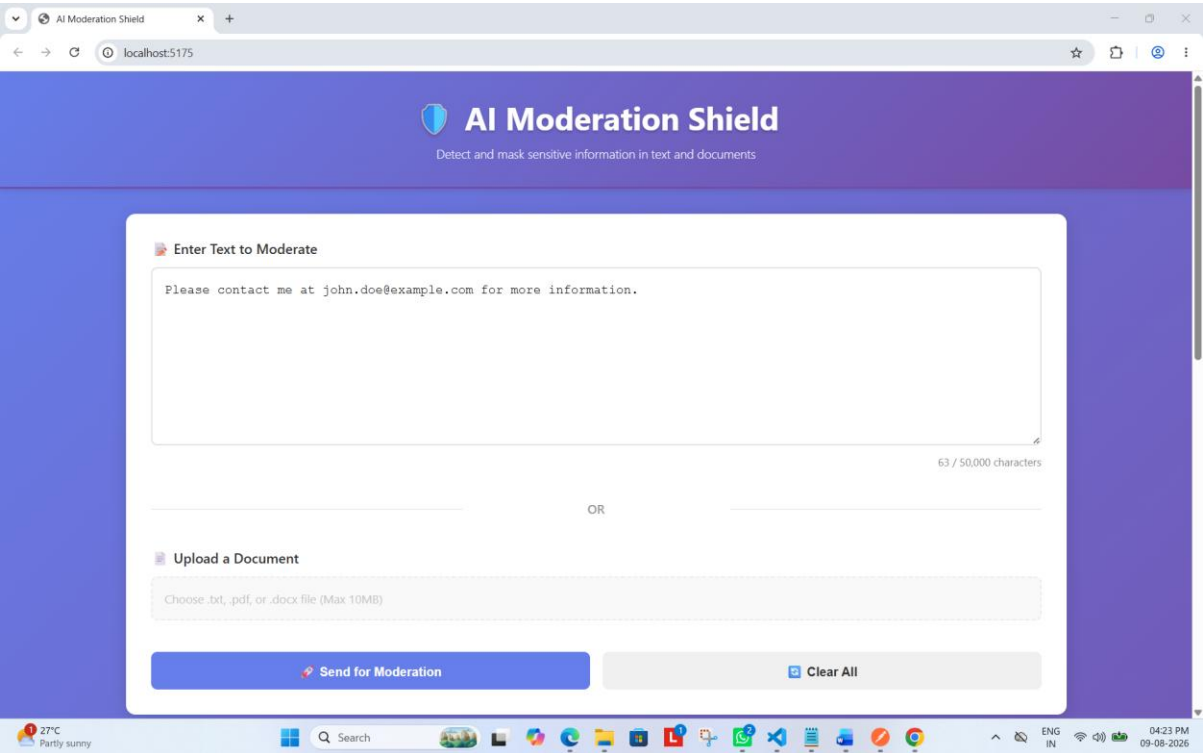
Hello, how are you today? This is a safe and clean message.

Copy Download

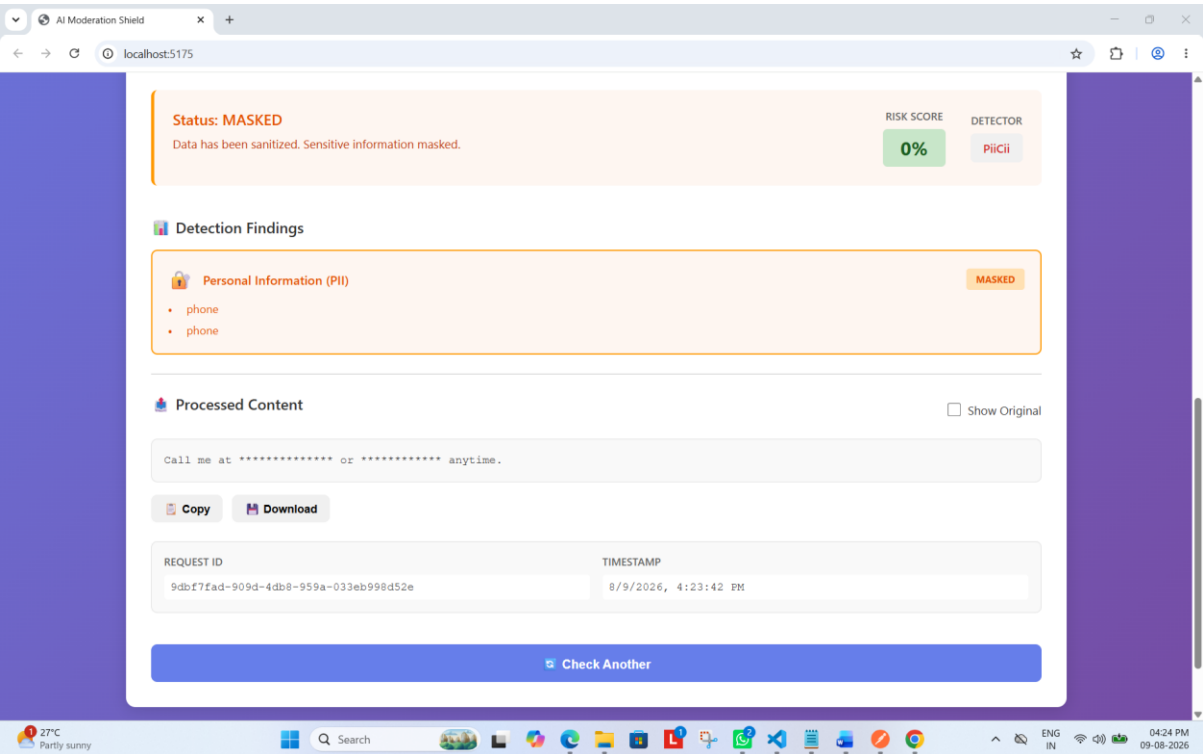
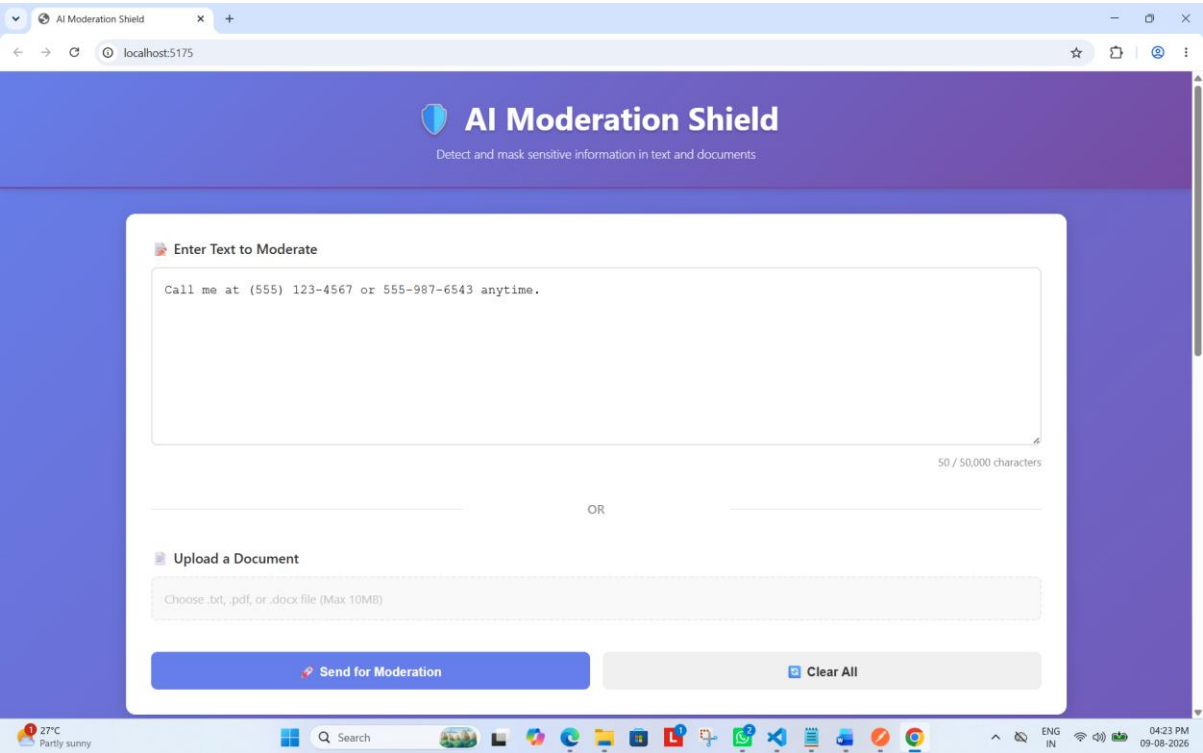
REQUEST ID	TIMESTAMP
f7795462-76bb-4800-9ab6-04421b004a29	8/9/2026, 4:22:23 PM

Check Another

Email –



Phone –



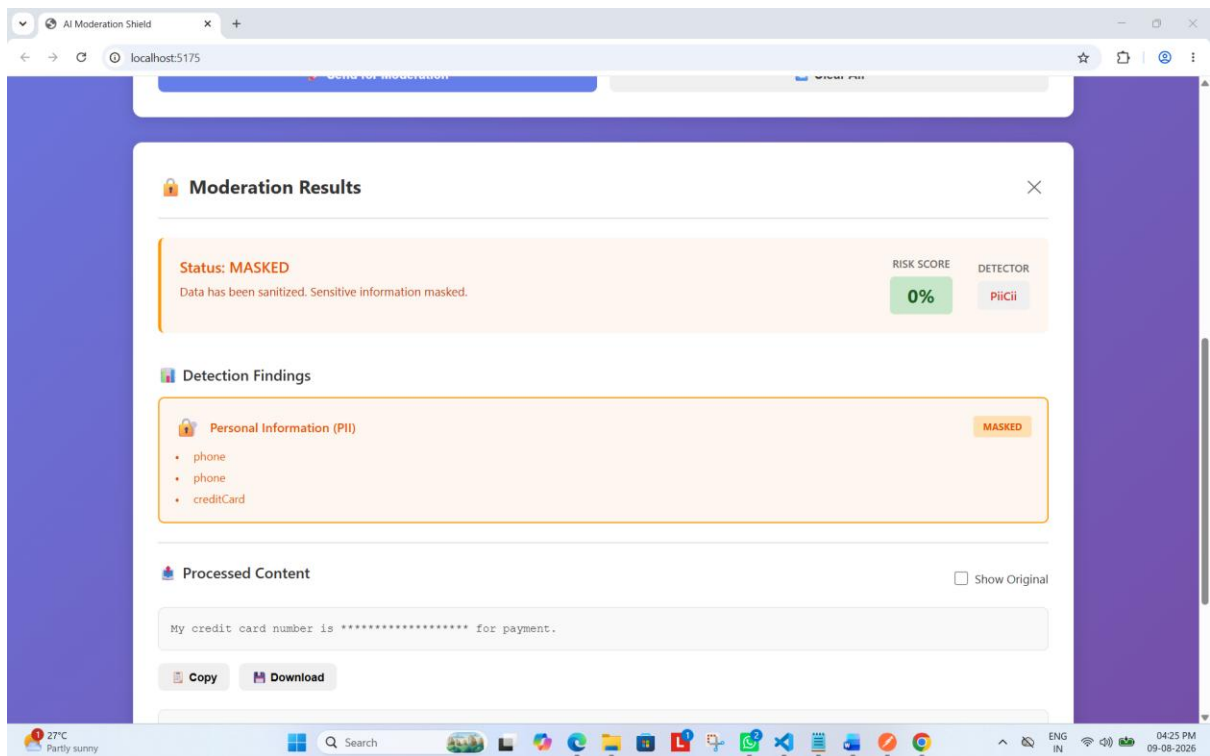
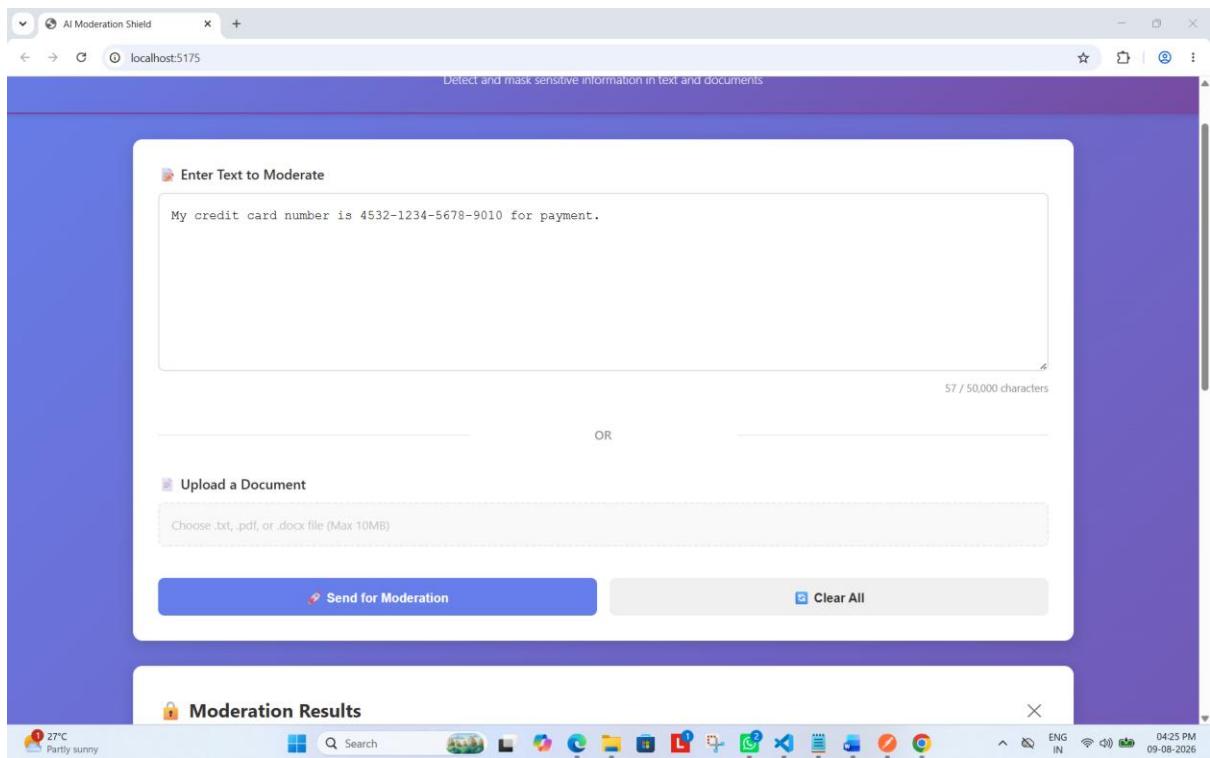
SSN

The screenshot shows the 'AI Moderation Shield' web application in a browser window. The page has a purple header with the title and a subtitle 'Detect and mask sensitive information in text and documents'. Below the header, there are two main input sections: 'Enter Text to Moderate' and 'Upload a Document'. The 'Enter Text to Moderate' section contains a text area with the text 'My social security number is 123-45-6789 for verification purposes.' and a character count '67 / 50,000 characters'. The 'Upload a Document' section has a dashed border indicating a file upload area with the text 'Choose .txt, .pdf, or .docx file (Max 10MB)'. At the bottom of the input section, there are two buttons: 'Send for Moderation' (blue) and 'Clear All' (grey). The browser's taskbar at the bottom shows the date and time as 04:24 PM on 09-08-2026.

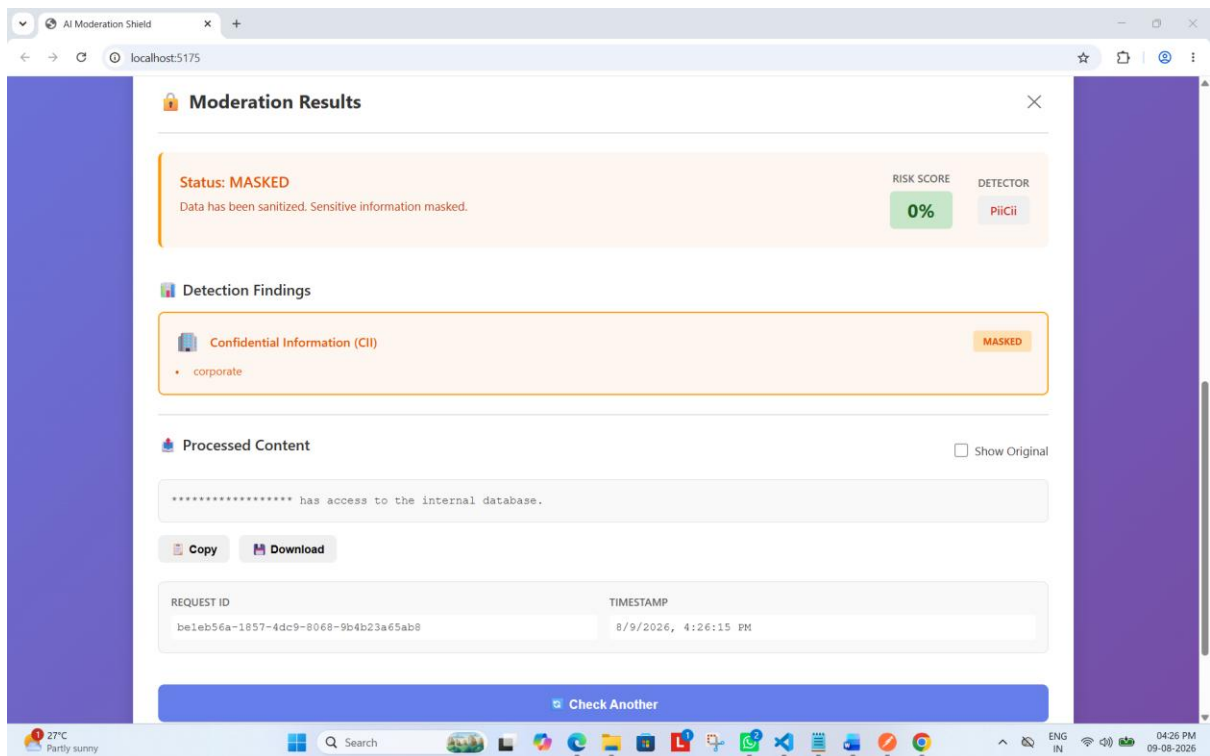
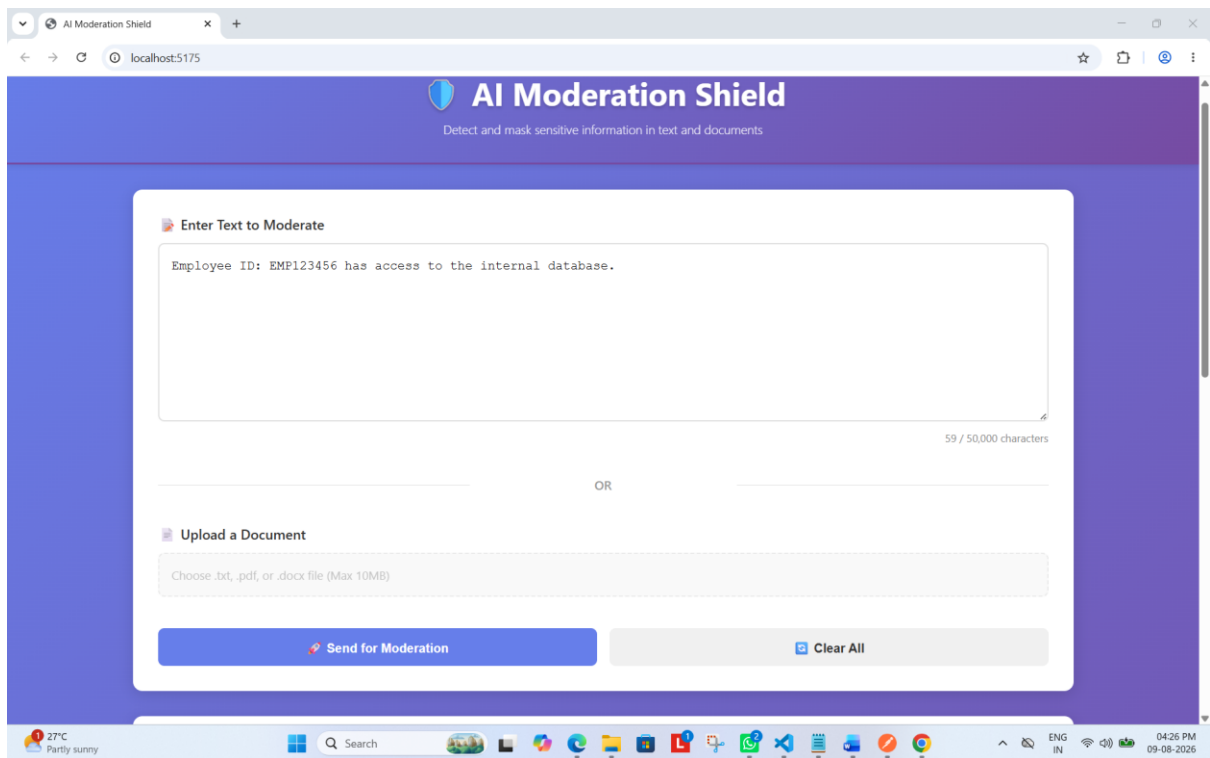
The screenshot shows the 'AI Moderation Shield' web application displaying the results of a moderation request. At the top, a red box indicates the 'Reason for Blocking': 'SSN detected in input. Request blocked.' Below this, the 'Detection Findings' section shows 'Personal Information (PII)' as the detected category, with a 'BLOCKED' status. The 'Processed Content' section shows '[No content to display]' and a 'Show Original' checkbox. Below this, there are 'Copy' and 'Download' buttons. At the bottom, a table displays the 'REQUEST ID' and 'TIMESTAMP' for the processed content. The 'REQUEST ID' is '55b9e80b-8803-4d06-84c2-f055d4a22ef3' and the 'TIMESTAMP' is '8/9/2026, 4:24:35 PM'. A 'Check Another' button is located at the bottom of the results section. The browser's taskbar at the bottom shows the date and time as 04:25 PM on 09-08-2026.

REQUEST ID	TIMESTAMP
55b9e80b-8803-4d06-84c2-f055d4a22ef3	8/9/2026, 4:24:35 PM

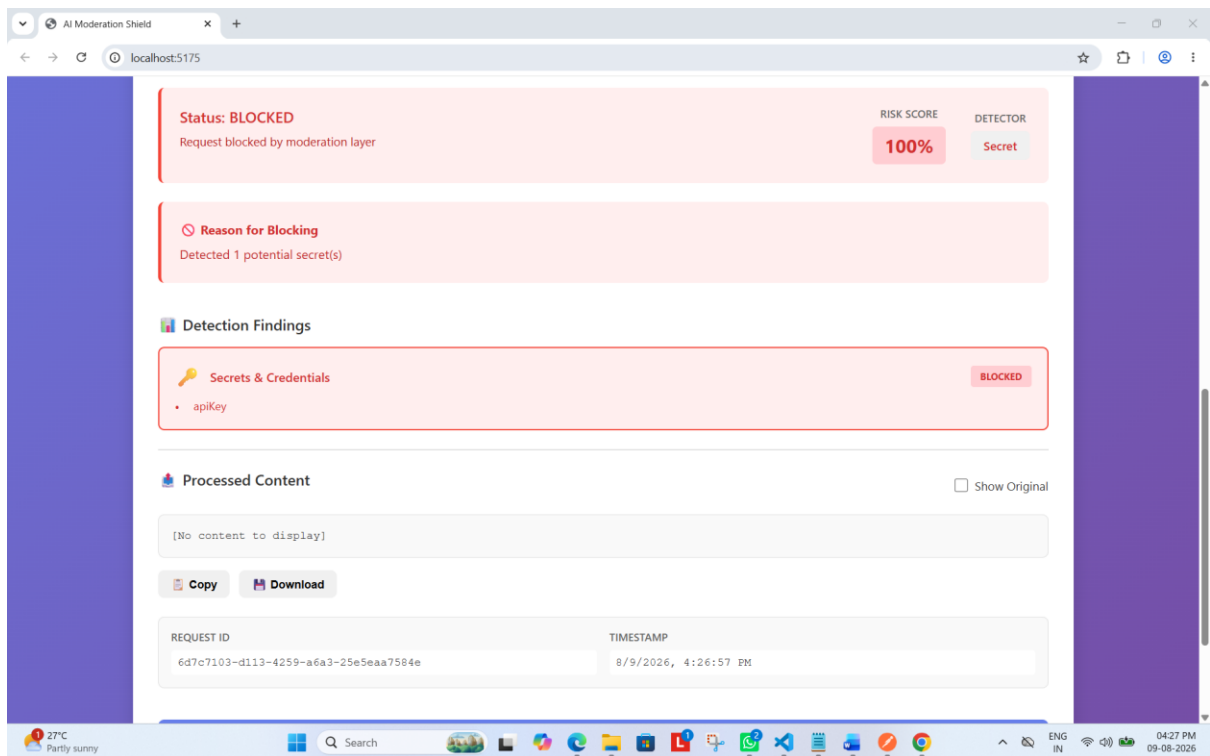
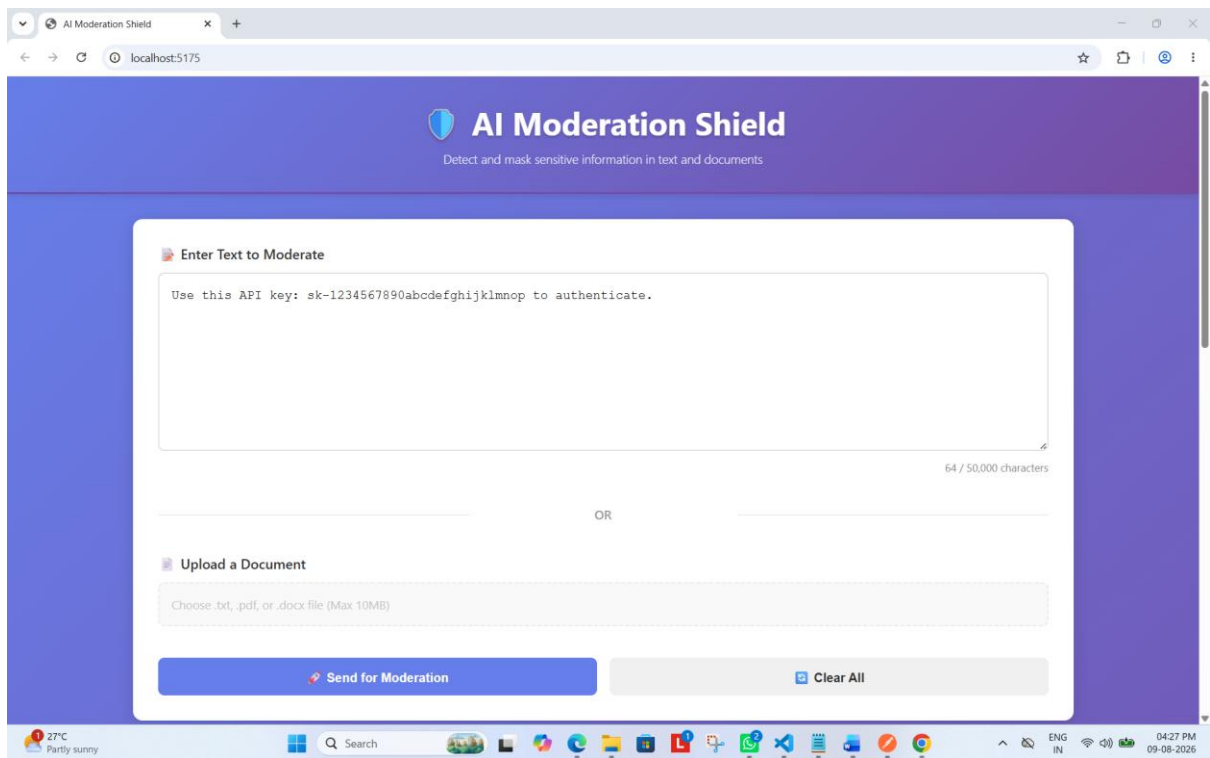
Credit card



Employee details



API key –



Aws key –

